Sistema de Agendamento de Consultas Médicas

Modelagem de Software — APS1

Carlos Eduardo Freire da Silva **Gabriel Henrique Lourenço Dias** **Kayque Vicente Silva**
Luiz Pedro Pereira Mendes Saraiva **Yury Nogueira de Oliveira Silva**

Instituto de Informática (INF) — Universidade Federal de Goiás (UFG)
Bacharelado em Engenharia de Software — Prof. Me. Diogo Oliveira Santos

2026/2

INF
UFG

UFG
UNIVERSIDADE
FEDERAL DE GOIÁS

Sumário

1. Visão Geral e o Problema da Clínica

2. Diagrama de Atividades (DA-01)

3. Diagrama de Sequência (DS-01)

4. Diagrama de Classes (DC)

5. Conclusão e Apêndices

Visão Geral e o Problema da Clínica

O Problema da Clínica e o que o Sistema Resolve

Problemas da rotina manual (Relatório, Seção 1.2):

- Recepção sobrecarregada: secretários perdem tempo atendendo ligações para marcar e desmarcar.
- Marcações duplas: sem uma agenda centralizada, dois pacientes acabam marcados no mesmo horário.
- Alto número de faltas: pacientes esquecem da consulta por falta de lembretes práticos.
- Perda de histórico: controles em papel dificultam remanejar horários e não deixam rastros para auditoria.

Como o sistema resolve na prática:

- Base única de horários: grade médica dividida em blocos de 30 minutos com consulta em tempo real.
- Reserva com trava atômica: no momento da confirmação, o slot é travado no banco na hora.
- Avisos automáticos: confirmação imediata e lembrete 24 h antes por e-mail e WhatsApp em paralelo.
- Auditoria imutável: registra autor, data/hora, IP e motivo obrigatório para cancelar ou remarcar (LGPD).

Quem Usa o Sistema: Os 5 Atores e suas Responsabilidades

Atores humanos (Relatório, Seção 1.3):

- Paciente: faz autoatendimento pelo navegador, escolhe médico e horário, agenda sozinho e pode cancelar ou remarcar com pelo menos 2 h de antecedência.
- Médico: cadastra sua disponibilidade semanal em slots de 30 min e marca o atendimento (início, fim ou falta). Não agenda nem cancela consultas.
- Secretário: acompanha as agendas, confirma consultas pendentes e pode agendar ou desmarcar para pacientes que ligam na recepção.
- Administrador: cadastra médicos, secretários e ativa especialidades. Não opera consultas.

Ator secundário:

- Sistema de Notificação: componente em segundo plano que dispara e-mails e mensagens de WhatsApp sem travar o uso da aplicação.

Regra de ouro dos perfis:

- Cada usuário tem exatamente um perfil exclusivo no sistema (`Perfil`), garantindo que o médico atenda pacientes mas não altere agendas alheias.

Como o Sistema Foi Dividido: Os 6 Módulos da Aplicação

Módulos de Acesso e Grade (Relatório, Seção 1.4):

- Cadastro e Autenticação: exige e-mail único para todos, CPF para pacientes e CRM/UF para médicos. Senha com hash criptográfico e bloqueio após erros seguidos.
- Consulta e Especialidades: catálogo público para o paciente buscar especialidades ativas e médicos credenciados.
- Agenda do Médico: organiza a grade em blocos padrão de 30 min (`LIVRE`, `BLOQUEADO` ou `OCUPADO`) entre 30 min e 90 dias no futuro.

Módulos de Operação e Mensagens:

- Agendamento: guia o paciente em 4 passos, faz o lock atômico da vaga e trata cancelamentos e remarcações com motivo obrigatório.
- Confirmação (Recepção): painel para o secretário contatar o paciente e registrar confirmação (opcional: médico atende mesmo sem isso).
- Notificação e Auditoria: fila de envio com retentativas automáticas e log imutável de todas as ações críticas.

Ciclo de Vida da Consulta: Os Estados e as Regras de Mudança

Como o estado muda (Quadro 1, Seção 1.5):

Estado Inicial	Ação / Evento	Próximo Estado
—	agendar	`AGENDADA`
`AGENDADA`	confirmar (opcional)	`CONFIRMADA`
`AGEND./CONF.`	médico chama na sala	`EM_ANDAMENTO`
`EM_ANDAMENTO`	médico conclui atendimento	`REALIZADA`
`AGEND./CONF.`	cancelar ($\geq$ 2 h)	`CANCELADA`
`AGEND./CONF.`	remarcar ($\geq$ 2 h)	`REMARCADA`
`AGEND./CONF.`	paciente não compareceu	`NAO_REALIZADA`

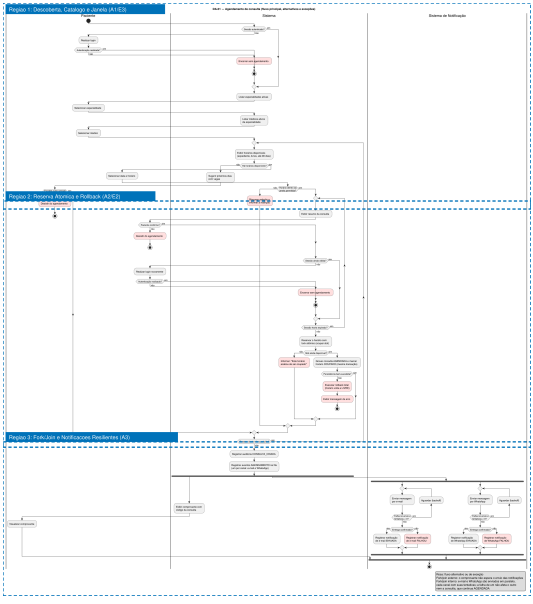
Regras estritas de cancelamento (RN-20):

- Prazo mínimo de 2 horas: se faltar menos de 2 h para o horário marcado, o sistema bloqueia o cancelamento ou remarcação.
- Motivo obrigatório: o paciente ou secretário deve justificar a ação por escrito para gerar histórico.
- Liberação imediata: no cancelamento, o slot volta na hora para `LIVRE`; na remarcação, a antiga é cancelada e cria-se uma nova.

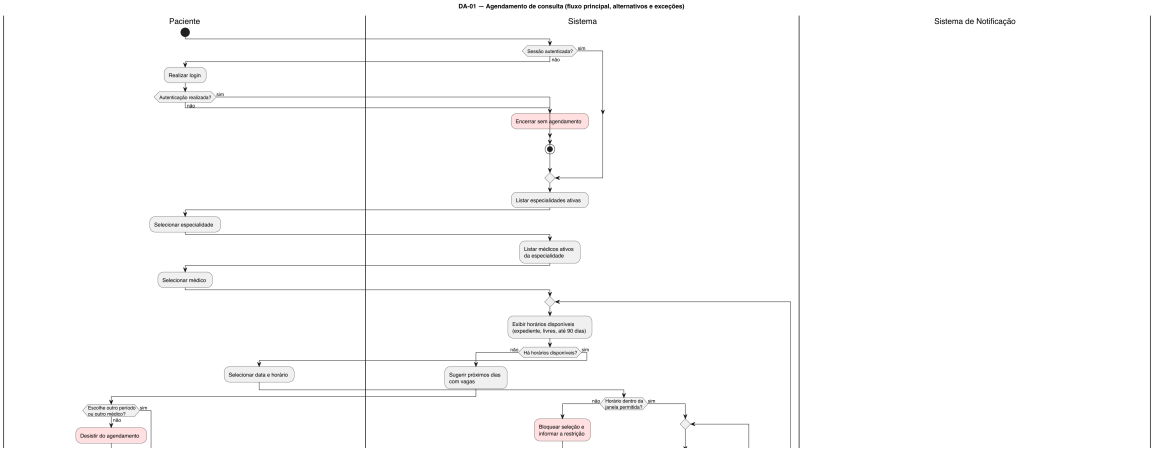


Diagrama de Atividades (DA-01)

DA-01 — Visão Geral do Fluxo de Agendamento (Completo)



DA-01 (Fase 1) — Escolha do Médico, Horários Livres e Janela



O Início do Agendamento: Filtros na Grade e Validação da Janela

O que o paciente faz (Relatório, Seção 2.1):

- Faz login com e-mail e senha (se já estiver logado, o fluxo segue direto).
- Escolhe a especialidade ativa no catálogo e em seguida escolhe o médico.
- O sistema carrega os horários livres dentro do expediente daquele médico (até 90 dias).

Decisão de modelagem:

- Filtro em cascata na raia do `Sistema` para não misturar médicos de áreas diferentes.

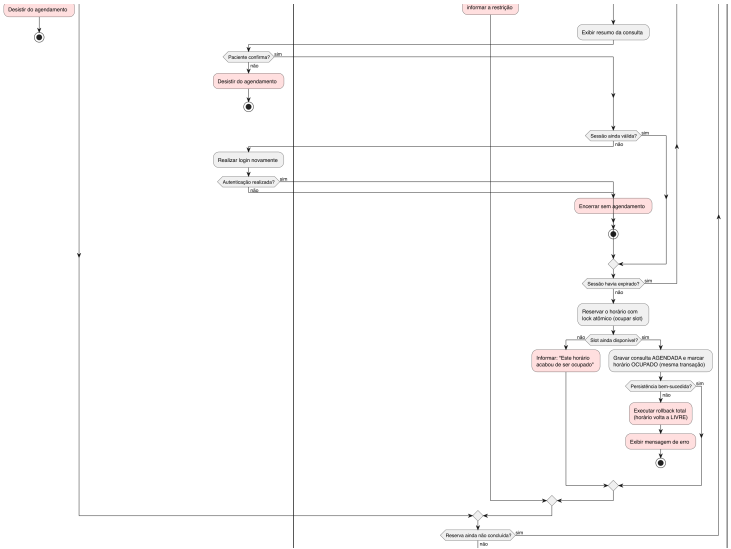
O que acontece se algo der errado:

- Sem vagas (Fluxo A1): o sistema sugere as próximas datas com horários livres; o paciente pode tentar outro período ou desistir.
- Fora da janela (Fluxo E3): se o paciente tentar marcar com menos de 30 min ou além de 90 dias (RN-13), o sistema bloqueia e explica a regra.

Evidência no diagrama:

- Nós de decisão com caminhos em rosa (A1 e E3) e retorno para a grade.

DA-01 (Fase 2) — Resumo, Lock Atômico do Horário e Rollback



Reserva da Vaga: Concorrência e Tratamento de Erros

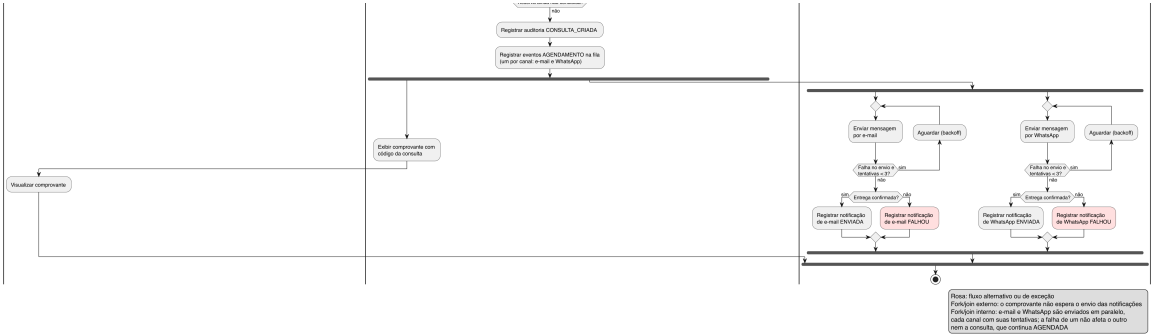
Prevenção de Concorrência (RN-08 e RN-10):

- Dois pacientes podem ver a mesma vaga ao mesmo tempo. Se a checagem não for atômica, ambos confirmam o mesmo horário.
- O sistema exibe o resumo; ao confirmar, aplica lock atômico no banco: ocupa o slot naquele milissegundo.
- Se a sessão expirou enquanto ele pensava, exige login mantendo os dados selecionados na tela.

Tratamento dos desvios (caminhos em rosa):

- Vaga tomada por outro (Fluxo A2): o sistema avisa "Este horário acabou de ser ocupado" e volta para a escolha de horários.
- Falha ao gravar no banco (Fluxo E2): faz rollback total; o horário volta a ficar `LIVRE` e avisa o erro na tela.
- Evidência no diagrama: decisões "Slot ainda disponível?" e "Persistência bem-sucedida?".

DA-01 (Fase 3) — Comprovante em Tela e Notificações Assíncronas



Comprovante e Mensagens: Por que o Envio é em Paralelo

O problema de rede que precisava resolver:

- Provedores de e-mail e APIs de WhatsApp podem oscilar ou demorar segundos. O paciente não pode ficar com a tela travada esperando isso.

Como o sistema resolve com bifurcação (fork/join):

- Ao gravar a consulta, o fluxo se divide em duas partes que andam ao mesmo tempo:
- Ramo 1: O paciente vê o comprovante com o código da consulta na hora na tela.
- Ramo 2: O `Sistema de Notificação` cuida do envio em segundo plano.

Envio resiliente por canal (Fluxo A3, RN-25 a RN-28):

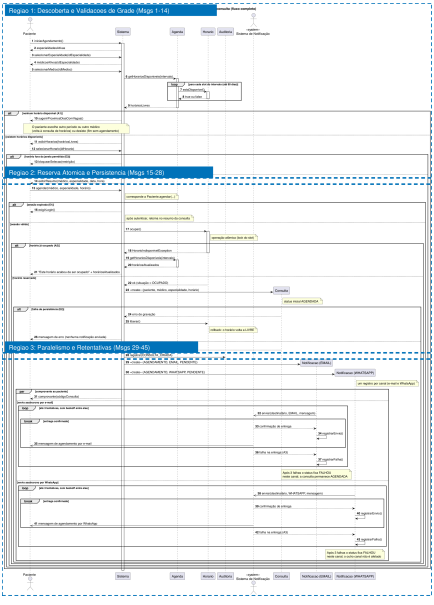
- E-mail e WhatsApp disparam em paralelo; cada canal tenta até 3 vezes com tempo de espera (backoff).
- Se o WhatsApp falhar após 3 tentativas, ele fica marcado como `FALHOU`, mas o e-mail continua e a consulta permanece `AGENDADA`.

Evidência no diagrama:

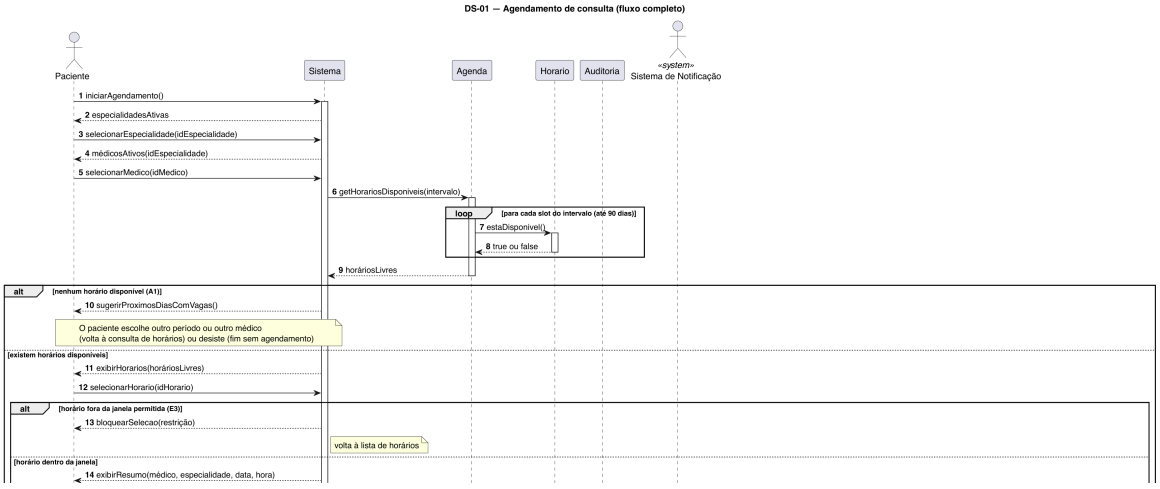
- Barras de fork/join e laços com cláusula `backward:Aguardar`.

Diagrama de Sequência (DS-01)

DS-01 — Visão Geral das 45 Mensagens entre Objetos (Completo)



DS-01 (Msgs 1 a 14) — Montagem da Grade e Desvios Iniciais



Troca de Mensagens Inicial: Como a Grade é Montada

O papel do participante 'Sistema' (Relatório, Seção 3.1):

- Funciona como controlador da aplicação. Ele recebe as ações da tela e delega para as classes de negócio, sem misturar detalhes visuais com o domínio.

Como as mensagens circulam (Msgs 1 a 9):

- `Paciente` chama `iniciarAgendamento()`; o `Sistema` retorna especialidades e médicos ativos.
- Ao selecionar o médico, o `Sistema` consulta os horários da `Agenda`.
- A `Agenda` roda um laço (`loop`) chamando `Horario.estaDisponivel()` para cada slot.

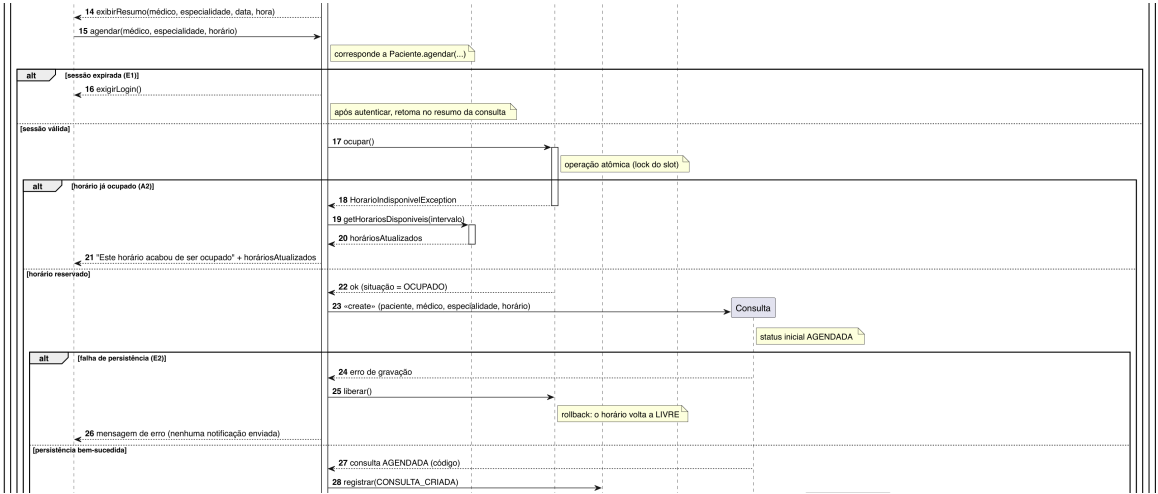
Desvios representados:

- Sem vagas (Msg 10 / A1): o `Sistema` sugere novas datas com vagas ao paciente.
- Fora da janela (Msg 13 / E3): `Sistema` chama `bloquearSelecao(restrição)`.
- Tudo certo (Msg 14): `Sistema` entrega `exibirResumo(...)`.

Evidência no diagrama:

- Mensagens numeradas 1 a 14 e fragmentos `loop` e `alt`.

DS-01 (Msgs 15 a 28) — Operação Horario.ocupar() e Rollback



A Confirmação no Código: Chamada ocupar() e Rollback

Como a trava atômica funciona no código (Msgs 17 e 18):

- O paciente chama `agendar(...)`. O `Sistema` chama direto `Horario.ocupar()`.
- Se outro paciente chegou primeiro, o método lança na hora a exceção `HorarioIndisponivelException`.
- O sistema captura a exceção, recarrega a grade e avisa o paciente (A2). Não há brecha de tempo entre checar e ocupar.

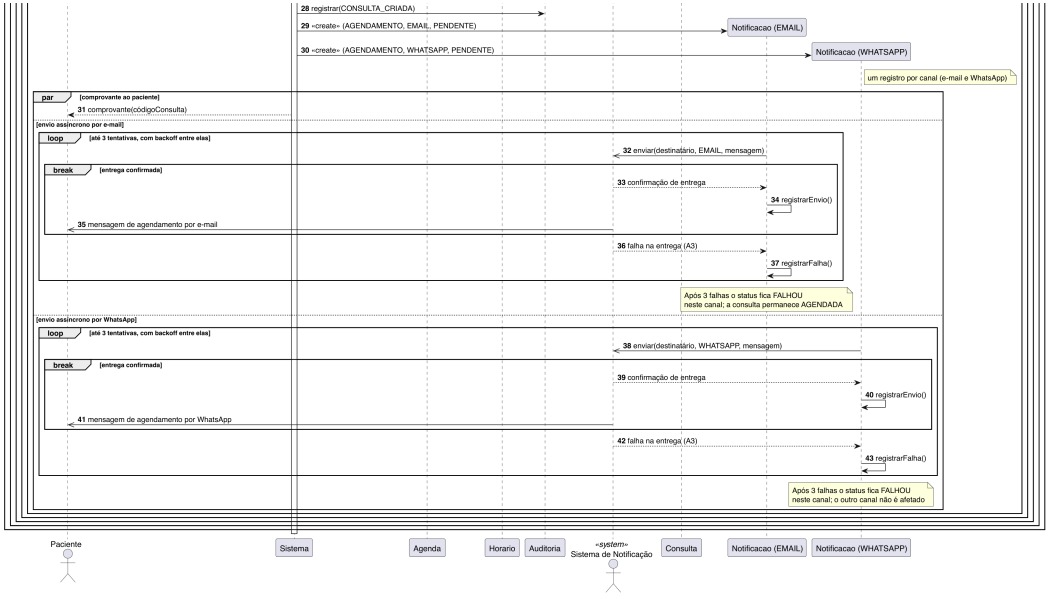
Criação da consulta e rollback em erro (Msgs 23 a 28):

- A `Consulta` só é instanciada via `«create»` depois que o horário já foi ocupado com sucesso.
- Se a gravação no banco der erro (E2), o `Sistema` chama `Horario.liberar()`; o horário volta a ser `LIVRE` e nada é enviado.
- Se gravou com sucesso, chama `Auditoria.registrar(CONSULTA_CRIADA)`.

Evidência no diagrama:

- Chamadas síncronas com retorno tracejado e blocos `alt/else`.

DS-01 (Msgs 29 a 45) — Paralelismo (par) e Retentativas com Backoff



Envio das Mensagens no Código: Bloco par e Retentativas

Criação dos objetos de notificação (Msgs 29 e 30):

- O sistema cria duas instâncias de `Notificacao` com status inicial `PENDENTE`: uma para `EMAIL` e outra para `WHATSAPP`.

Por que usamos o bloco 'par' e setas abertas ('-»'):

- O fragmento `par` executa três coisas ao mesmo tempo: entrega do comprovante (Msg 31), envio por e-mail e envio por WhatsApp.
- As setas abertas indicam chamadas assíncronas: o sistema dispara para o `Sistema de Notificação` e libera o paciente na hora.

Laços de repetição com 'break' (Msgs 32 a 45):

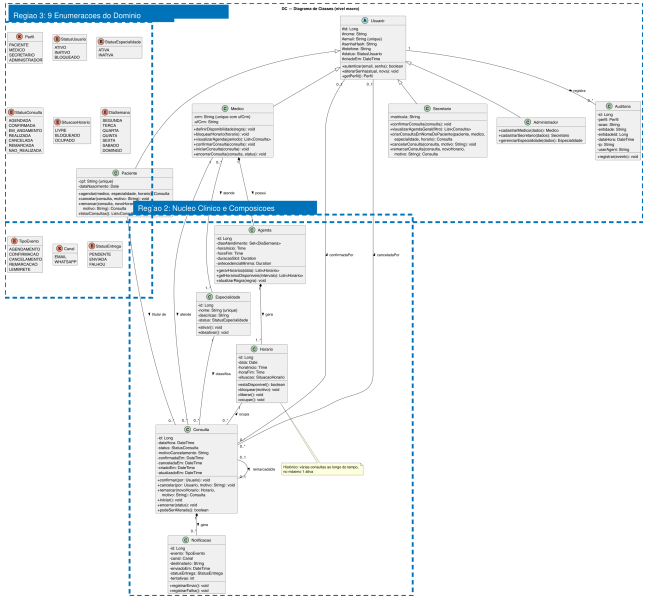
- Cada canal roda seu próprio laço com até 3 tentativas e tempo de espera entre elas.
- Se a entrega for confirmada, o `break` encerra o laço daquele canal.
- Se falhar 3 vezes, fica como `FALHOU`, sem afetar a consulta nem o outro canal.

Evidência no diagrama:

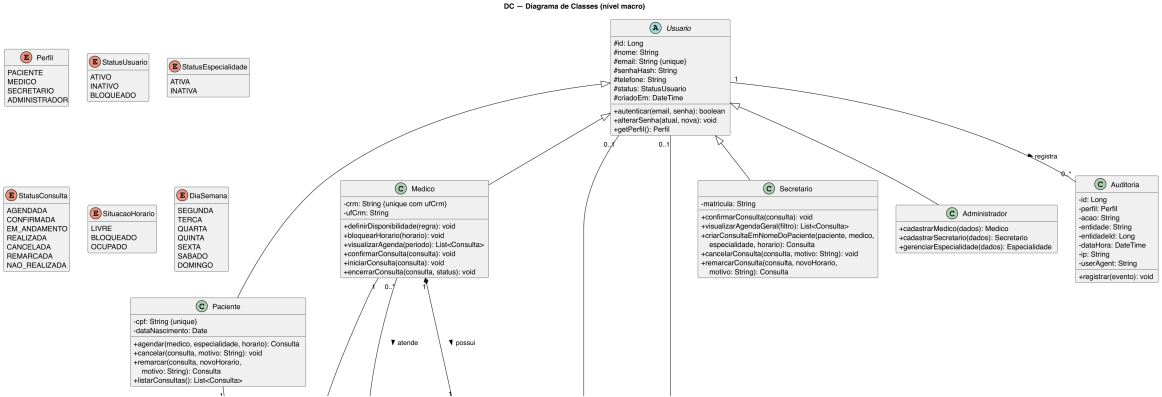
- Fragmentos `par`, `loop`, `break` e setas abertas (`-»`).

Diagrama de Classes (DC)

DC — Visão Geral das 11 Classes e 9 Enumerações (Completo)


DC — Diagrama de Classes (visão macro)
Regiao 3: 9 Enumeracoes do Dominio
Perfil
PACIENTE
MEDICO
SECRETARIO
ADMINISTRADOR
StatusUsuario
ATIVO
INATIVO
BLOQUEADO
StatusEspecialidade
ATIVA
INATIVA
StatusConsulta
AGENDADA
CONFIRMADA
RIC_AGENDAMENTO
EM_ATENDIMENTO
CANCELADA
REMARCADA
MAU_REALIZADA
DiaSemana
SEGUNDA
TERCA
QUARTA
QUINTA
SEXTA
SABADO
DOMINGO
BloqueioHorario
LIVRE
BLOQUEADO
OCUPADO
TipoEvento
AGENDAMENTO
CONFIRMACAO
CANCELAMENTO
REMARCACAO
LEMBRETE
Canal
EMAIL
WHATSAPP
StatusEntrega
PENDENTE
ENVIADO
FALHOU
Usuario
-id: Long
-lnome: String
-lemail: String (unique)
-lsenhaHash: String
-lultimoLogin: DateTime
-lstatus: StatusUsuario
-lcriadoEm: DateTime
+autenticar(email, senha): boolean
+bloquear(): void
+getPerfil(): Perfil
Medico
-crm: String (unique com ufCrm)
-ufCrm: String
+definirDisponibilidade(regra): void
+bloquearHorario(data, hora): void
+visualizarAgenda(periodo): List<Consulta>
+confirmarConsulta(consulta): void
+cancelarConsulta(consulta): void
+atualizarStatusConsulta(consulta, status): void
Secretaria
-matricula: String
+confirmarConsulta(consulta): void
+visualizarAgendaGeral(filtro): List<Consulta>
+cadastrarPaciente(dados): Paciente
-especialidade, horario): Consulta
+cancelarConsulta(consulta, motivo: String): void
+remarcarConsulta(consulta, novoHorario,
-motivo: String): Consulta
Administrador
+cadastrarMedico(dados): Medico
+cadastrarSecretaria(dados): Secretaria
+gerenciarEspecialidades(dados): EspecialidadeAuditoria
-id: Long
-quem: Ator
-acao: String
-entidade: String
-idEntidade: Long
-detalhes: JsonObject
-ip: String
-userAgent: String
+registrarEvento(): void
Paciente
-cpf: String (unique)
-dataNascimento: Data
+agendarConsulta(medico, especialidade, horario): Consulta
+cancelarConsulta(consulta, motivo: String): void
+remarcarConsulta(consulta, novoHorario,
-motivo: String): Consulta
+listarConsultas(): List<Consulta>
Regiao 2: Nucleo Clinico e Composicoes
Agenda
-id: Long
-diasAtendimento: Set<DiaSemana>
-horaInicio: Time
-horaFim: Time
-duracaoSlot: Duration
-slotsOcupadosEmHora: Map<Slot
+gerarHorariosVazios(): List<Horario>
+definirIntervaloDiario(inicio, fim): void
+aplicarRegra(regra): void
Especialidade
-id: Long
-nome: String (unique)
-duracaoMin: int
-status: StatusEspecialidade
+ativar(): void
+desativar(): void
Horario
-id: Long
-data: Date
-horaInicio: Time
-horaFim: Time
-situacao: SituacaoHorario
+estaDisponivel(): boolean
+bloquear(motivo): void
+liberar(): void
+ocupar(): void
Consulta
-id: Long
-criadaEm: DateTime
-status: StatusConsulta
-motivoCancelamento: String
-canceladaPor: String
-canceladaEm: DateTime
+statusEm: DateTime
-atualizadoEm: DateTime
+confirmar(ator : Usuario): void
+cancelar(ator : Usuario, motivo: String): void
+remarcar(novoHorario: Horario,
motivo: String): Consulta
+iniciar(): void
+encerrar(status): void
+podeSerAlterada(): boolean
Notificacao
-id: Long
-evento: TipoEvento
-canal: Canal
-destinatario: String
-assunto: String (null
-statusEntrega: StatusEntrega
-tentativas: int
+registrarEnvio(): void
+registrarFalha(): void
Historico: versao consultas ao longo do tempo, no relatorio 1 abrir
agenda
1
1..*
1
0..*
1
0..*
1
0

DC (Perfis) — Herança a Partir da Classe Abstrata Usuario



A Hierarquia de Perfis: Por que Usamos Herança

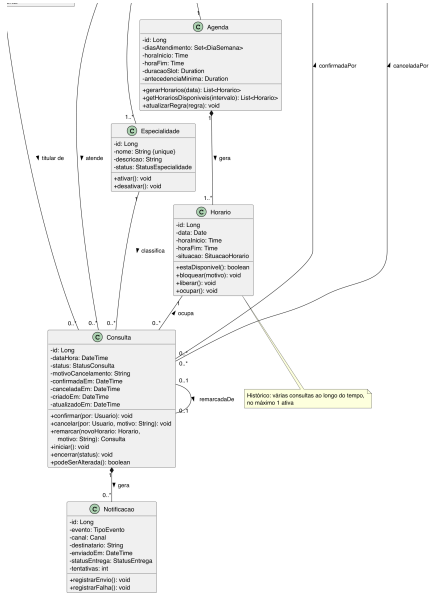
A classe abstrata ‘Usuario’ e atributos protegidos (#):

- Ninguém no sistema é só um usuário genérico; por isso ela é abstrata e não pode ser criada diretamente.
- Centraliza o que todos têm: `id`, `nome`, `email` único, `senhaHash`, `telefone` e `status`.
- Os atributos usam `#` (protegidos) para que as subclasses herdem e usem direto, sem expor métodos públicos perigosos.

O que cada perfil acrescenta de concreto (Relatório, Seção 4.1):

- `Paciente`: adiciona `cpf` único e métodos para `agendar()`, `cancelar()` e `remarcar()`.
- `Medico`: adiciona `crm` e `ufCrm`, e métodos para `definirDisponibilidade()` e atender consultas.
- `Secretario`: adiciona `matricula` e métodos para confirmar ou agendar por telefone.
- `Administrador`: métodos exclusivos para cadastrar médicos, secretários e especialidades.

DC (Clínica) — Composições, Histórico de Horários e Auditoria



Composições, Histórico de Horários e Auditoria

Composições no Modelo (Relatório, Seção 4.1):

- Médico → Agenda (1:1): a agenda pertence ao médico; sem o profissional, ela perde a razão de existir.
- Agenda → Horário (1:1..*): os blocos de 30 min pertencem exclusivamente à agenda que os gerou.
- Consulta → Notificação (1:0..*): as mensagens são filhas diretas dos eventos daquela consulta.

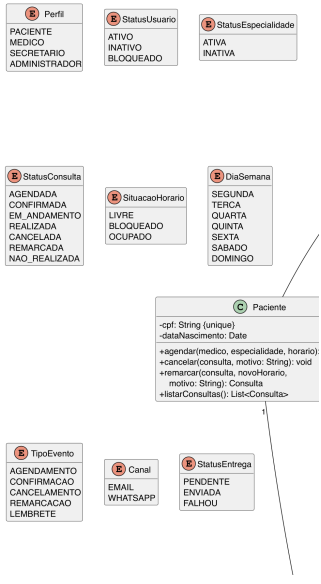
Exceção: Associação Simples (LGPD):

- Usuário – Auditoria: propositalmente não é composição para preservar os logs mesmo se a conta for excluída.

Relação Horário → Consulta (1 para 0..*):

- Um mesmo horário guarda o histórico de todas as consultas que passaram por ele, mas admite no máximo uma consulta ativa.
- Autoassociação `remarcadaDe` (0..1 para 0..1): a nova consulta aponta para a consulta cancelada anterior.
- Rastreabilidade: associações `confirmadaPor` e `canceladaPor` com `Usuario`.

DC (Enumerações) — Os 9 Tipos Enumerados do Domínio



Enumerações e Controle dos Estados do Sistema

Por que usamos 9 enumerações em vez de texto livre:

- Elimina erros de digitação e impede estados inválidos no banco de dados e no código da aplicação.

O que cada enumeração controla (Relatório, Seção 4.1):

- `Perfil`: trava as contas em `PACIENTE`, `MEDICO`, `SECRETARIO` ou `ADMINISTRADOR`.
- `StatusUsuario`: `ATIVO`, `INATIVO` ou `BLOQUEADO` (após sucessivas senhas erradas).
- `StatusEspecialidade`: `ATIVA` ou `INATIVA`.
- `DiaSemana`: parametriza os dias de atendimento na agenda do médico.

Estados da grade e do atendimento:

- `SituacaoHorario`: `LIVRE`, `BLOQUEADO` (pelo médico) ou `OCUPADO` (com consulta).
- `StatusConsulta`: os 7 estados do ciclo (`AGENDADA`, `CONFIRMADA`, `EM_ANDAMENTO`, `REALIZADA`, `CANCELADA`, `REMARCADA`, `NAO_REALIZADA`).
- `TipoEvento`: gatilho do disparo de mensagens.
- `Canal`: limita a `EMAIL` ou `WHATSAPP`.
- `StatusEntrega`: `PENDENTE`, `ENVIADA` ou `FALHOU`.

Conclusão e Apêndices

O que Está Especificado nos Apêndices do Trabalho

Apêndice A — Requisitos e Regras de Negócio:

- 21 Requisitos Funcionais (RF): detalham tela por tela o que o sistema deve fazer (cadastro, busca, grade, reserva, confirmação e alertas).
- 28 Regras de Negócio (RN): definem as travas reais (antecedência $\geq$ 30 min para agendar, limite de 90 dias, prazo de 2 h para cancelar com motivo).
- Todos os nós e mensagens dos diagramas vieram diretamente dessas regras documentadas.

Apêndice B — Códigos PlantUML dos Diagramas:

- Fontes originais em texto: arquivos completos dos diagramas DA-01, DS-01 e DC.
- Controle de versão no Git: permite versionar a modelagem junto com o código do projeto.
- Conformidade ABNT NBR 14724: documentação técnica que garante reprodutibilidade total.

Como os Três Modelos se Conectam na Prática

O papel de cada diagrama no projeto:

- DA-01 (Atividades): mostrou a jornada do paciente na tela, onde ele toma decisões e o que acontece quando algo dá errado (vaga ocupada, sessão expirada).
- DS-01 (Sequência): mostrou como o código resolve isso em tempo de execução, chamando `ocupar()` de forma atômica e disparando notificações em paralelo.
- DC (Classes): amarrou tudo na estrutura de objetos, com tipos fortes nas 9 enumerações, composições onde o ciclo de vida depende e herança protegida.

O que a modelagem garantiu antes de codificar:

- Sem corrida crítica: conflitos de dois pacientes na mesma vaga foram eliminados no modelo.
- Tolerância a falhas: lentidão de redes externas não derruba o agendamento.
- Rastreabilidade total: conformidade com a LGPD prevista desde a concepção das classes.

Obrigado!

Sistema de Agendamento de Consultas Médicas

Modelagem de Software – APS 1 | UFG / INF (2026/2)

Dúvidas e Discussão

ENGENHARIA DE SOFTWARE

INF
INSTITUTO DE INFORMÁTICA

UFG
UNIVERSIDADE FEDERAL DE GOIÁS